

Assignment 3: The Great M&M Count

Fri Sep 11 2026

When you open a bag of M&M's, are the colors random?

Let's investigate this question. Each student will open a bag of M&M's, count candies by color, and contribute to a class dataset. Use NumPy, pandas, and a Jupyter notebook to look for patterns.

You may eat the evidence, but only after your counts have been checked and recorded.

Learning goals

- Create and inspect a pandas `DataFrame`;
- Apply familiar NumPy operations to pandas data;
- Select, filter, group, summarize, and reshape data;
- Convert candy counts into proportions;
- Translate a research question into a pseudo-algorithm;
- Use evidence to evaluate a claim about randomness; and
- Communicate defensible conclusion.

1 Collect the evidence

Each student open one (or more) bags of M&M's. For each bag:

1. Assign the bag a unique identifier.
2. Sort the candies by color and count each color.
3. Calculate the total number of candies.
4. Confirm that the color counts add up to the total.
5. Enter the results into the shared class dataset.

bag_id	blue	brown	green	orange	red	yellow	total
Bag_01							
Bag_02							

Before anyone samples the evidence, verify

$$\text{blue} + \text{brown} + \text{green} + \text{orange} + \text{red} + \text{yellow} = \text{total}.$$

In a Markdown cell, briefly identify one thing that could go wrong during data collection and explain how you checked its work.

2 Decide what “random” means

The question sounds simple, but *random* can mean several things:

- Is every color equally likely?
- Does every bag have approximately the same color proportions?
- Could the differences among bags reasonably be explained by chance?
- Do larger bags appear more consistent than smaller bags?

In a Markdown cell:

1. Describe at least two possible meanings of “random.”
2. Select one meaning for your investigation.
3. Rewrite it as a specific, answerable question.
4. Make a prediction: *I predict that ... because ...*

You are not graded on whether your prediction is correct ;)

3 Plan before you code

Write a pseudo-algorithm describing the logic of your analysis in plain language. Adapt the following example to your question.

```
INPUT: M&M color counts collected by the class

1. Check the bag identifiers and color counts.
2. Confirm that each row total equals the sum of its colors.
3. Calculate the total number of candies of each color.
4. Convert color counts into proportions.
5. Compare individual bags with the complete class dataset.
6. Define how a bag will be judged typical or unusual.
7. Create a visualization of the most important result.
8. Use the evidence to answer the research question.

OUTPUT: Tables, a visualization, and a defensible conclusion.
```

Each major section of your notebook should connect to a step in your plan!

4 Meet the pandas DataFrame

Load the shared class data and inspect it.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
```

```

5 df = pd.read_csv("mnmm_class_data.csv")
6 df.head()

```

Use pandas to determine the number of bags, available columns, column types, missing values, and any impossible counts. Useful starting points include:

```

1 df.shape
2 df.info()
3 df.describe()
4 df.isna().sum()

```

Do not silently repair a questionable observation. Explain what you changed and why.

Now validate the recorded totals:

```

1 color_cols = ["blue", "brown", "green", "orange", "red", "yellow"]
2 df["calculated_total"] = df[color_cols].sum(axis=1)
3 df["total_matches"] = df["calculated_total"] == df["total"]
4
5 df.loc[~df["total_matches"]]

```

5 Reconnect with NumPy

Extract the bag totals as both a pandas **Series** and a NumPy array.

```

1 totals_series = df["total"]
2 totals_array = totals_series.to_numpy()
3
4 np.mean(totals_array)
5 np.median(totals_array)
6 np.min(totals_array)
7 np.max(totals_array)
8 np.std(totals_array)

```

Calculate equivalent summaries with pandas. Explain one similarity between a **Series** and an array, one useful feature pandas adds, and why standard deviations may differ unless the same degrees-of-freedom convention is used.

```

1 totals_series.mean()
2 totals_series.median()
3 totals_series.min()
4 totals_series.max()
5 totals_series.std(ddof=0) # Match NumPy's default

```

6 Count, summarize, and reshape

Calculate the class-wide color counts and proportions.

```

1 overall_counts = df[color_cols].sum()
2 grand_total = overall_counts.sum()

```

```

3 overall_proportions = overall_counts / grand_total
4
5 overall_summary = pd.DataFrame({
6     "count": overall_counts,
7     "proportion": overall_proportions
8 }).sort_values("proportion", ascending=False)
9
10 overall_summary

```

Create a bag-by-color proportion table:

```

1 bag_proportions = df[color_cols].div(df["total"], axis=0)
2 bag_proportions.index = df["bag_id"]
3 bag_proportions

```

Answer the following:

- Which color was most common? Which was least common?
- Which bag had the most candies? Which had the fewest?
- Did any bag look especially unusual?
- Does unusual necessarily mean nonrandom? Explain.

7 Choose your randomness path

Complete one of the following investigations.

A: Equal shares

Assume that all six colors are equally likely. Calculate the expected counts, compare them with the observed counts, and decide whether equal probability is a reasonable interpretation of “random.”

```

1 expected_proportion = 1 / len(color_cols)
2 expected_counts = pd.Series(
3     grand_total * expected_proportion,
4     index=color_cols
5 )
6 comparison = pd.DataFrame({
7     "observed": overall_counts,
8     "expected": expected_counts,
9     "difference": overall_counts - expected_counts
10 })
11 comparison

```

B: Build virtual bags

Treat the overall class proportions as the probabilities for a random bag. Select one real bag and simulate many bags of the same size.

```

1 rng = np.random.default_rng(123)
2
3 selected_bag = "Bag_01"
4 bag_size = int(df.loc[df["bag_id"] == selected_bag, "total"].iloc[0])
5
6 simulated_bags = rng.multinomial(
7     n=bag_size,
8     pvals=overall_proportions.loc[color_cols].to_numpy(),
9     size=10_000
10 )

```

Compare the real bag with the simulations. Is its composition typical or unusual? Explain how you decided.

C: Find the oddest bag

Measure the distance between each bag and the overall class proportions:

$$D_b = \sum_c |p_{b,c} - p_c|.$$

```

1 distance = bag_proportions.sub(overall_proportions, axis=1).abs().sum(axis=1)
2 distance.sort_values(ascending=False)

```

Rank the bags from most typical to most unusual. Explain why smaller bags might naturally receive larger scores.

D: Invent your own

Develop another defensible investigation. State what you are measuring, why it is relevant, what assumptions you are making, and how it helps answer your question.

8 Show us the evidence

Create at least one visualization that directly supports your investigation. For example:

```

1 mnm_colors = {
2     "blue": "royalblue", "brown": "saddlebrown", "green": "forestgreen",
3     "orange": "darkorange", "red": "firebrick", "yellow": "gold"
4 }
5
6 plot_colors = [mnm_colors[color] for color in overall_proportions.index]
7 overall_proportions.plot(kind="bar", color=plot_colors)
8 plt.title("Color Proportions in the Class M&M Dataset")
9 plt.xlabel("Color")
10 plt.ylabel("Proportion")
11 plt.ylim(0, 1)
12 plt.tight_layout()
13 plt.show()

```

Your chart must have an informative title, labeled axes, an appropriate legend when needed, and a short Markdown interpretation. Make it inviting, but remember: a beautiful chart should also tell the truth.

9 One more thing

Choose one extension:

- identify the two most similar bags;
- write a function that produces a color report for any bag;
- invent an alternative “oddness score”;
- compare two different definitions of random;

Explain what your extension adds to the investigation.

10 Conclusion

Write a short (about 200 word) conclusion addressing:

1. the specific version of randomness you investigated;
2. the evidence you found;
3. whether the evidence supported your prediction;
4. what you can and cannot conclude;
5. how the number and size of bags affect your result; and
6. what additional data would strengthen the investigation.

End with: *Based on this analysis, the strongest conclusion I can justify is ...*

Notebook checklist

- ☐ Runs from beginning to end without errors.
- ☐ Includes code and Markdown explanations.
- ☐ Follows the major steps of the pseudo-algorithm.
- ☐ Uses both NumPy and pandas.
- ☐ Includes at least one useful visualization.
- ☐ Uses a seed for any simulation.
- ☐ Includes a creative extension and an evidence-based conclusion.

Assessment

	Points
Data collection and validation	1
Question, prediction, and pseudo-algorithm	1
NumPy connection	1
pandas analysis	2
Investigation of randomness	1
Visualization	1
Creativity and critical thinking	2
Communication and reproducibility	1
Total	10